
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Corso di Elementi di Intelligenza Artificiale

MedExpert AI

Sistema Esperto di Diagnosi Medica

Implementazione in Prolog con Interfaccia Grafica Python

DOCENTE	STUDENTI
Prof. Giancarlo Sperli	Luciano Meccariello N46007465 Gaspere Tortora N46007942 Andrea Francesco Bruno N46007639

Anno Accademico 2025/2026

Indice

1	Obiettivo del Progetto
2	Il Dominio: Diagnosi Medica
3	Architettura del Sistema
4	Knowledge Base in Prolog
5	Regole di Inferenza
6	Interfaccia Grafica (Python / Tkinter)
7	Test e Validazione
8	Conclusioni

1. Obiettivo del Progetto

Il progetto realizza un **sistema esperto per la diagnosi medica** che combina una Knowledge Base in Prolog con un'interfaccia grafica in Python/Tkinter. L'obiettivo è mostrare in modo concreto i concetti del corso (rappresentazione della conoscenza, backward chaining, sistemi a regole) applicandoli a un dominio realistico.

1. Codificare in Prolog **17 patologie** con sintomi, categorie, descrizioni e trattamenti.
2. Implementare un motore **goal-driven** che ordini le diagnosi candidate per **Indice di Copertura Sintomatica (ICS)**.
3. Fornire una **explanation facility** che esponga sintomi trovati e mancanti.
4. Offrire una **GUI interattiva** usabile anche da utenti non tecnici.
5. Validare il sistema con una **suite di test automatizzati** in Prolog.

2. Il Dominio: Diagnosi Medica

La diagnosi medica è un dominio adatto ai sistemi esperti: il ragionamento clinico si formalizza naturalmente in regole del tipo «se i sintomi sono X, Y, Z allora la malattia candidata è W». Raramente un paziente presenta tutti i sintomi tipici di una patologia e gli stessi sintomi possono essere associati a malattie diverse: serve un **ranking** delle diagnosi, non una risposta binaria.

Per ogni malattia M della KB il sistema calcola un **Indice di Copertura Sintomatica**:

 formula

$$ICS(M) = \text{round}(|Sintomi_Paziente \cap Sintomi_M| / |Sintomi_M| \times 100)$$

L'ICS misura quale frazione dei sintomi tipici di M è presente nel paziente. Le diagnosi vengono presentate ordinate per ICS decrescente; la decisione clinica finale resta del medico. La KB copre 17 patologie su 10 categorie mediche:

Categoria	Patologie
Respiratorie	Influenza, COVID-19, Bronchite, Polmonite
Gastrointestinali	Gastrite, Appendicite, Reflusso gastroesofageo
Neurologiche	Emicrania, Meningite
Cardiovascolari	Ipertensione, Tachicardia
Altre	Anemia, Diabete tipo 2, Ipotiroidismo, Cistite, Allergia stagionale, Depressione

Tabella 1 — Patologie nella Knowledge Base (17 totali).

3. Architettura del Sistema

Il sistema è organizzato su tre livelli. La GUI Python raccoglie i sintomi dall'utente e costruisce le query; SWI-Prolog, lanciato in *subprocess*, esegue l'inferenza sulla KB e restituisce le diagnosi ordinate su stdout; Python interpreta l'output e popola la GUI. La scelta di subprocess (al posto di bridge come PySwip) elimina dipendenze esterne e rende il tutto portabile.

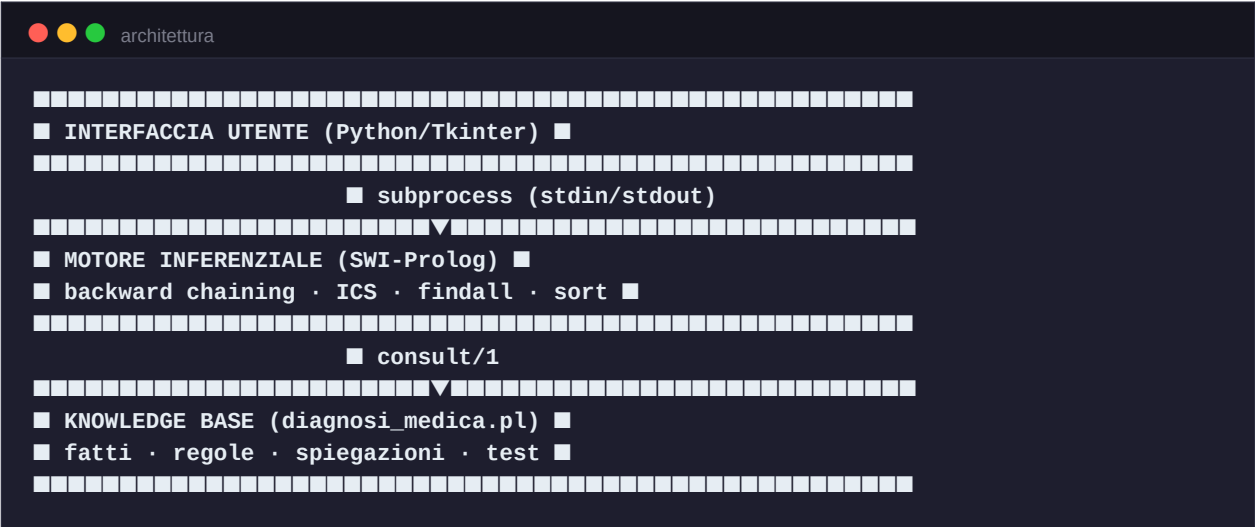


Figura 1 — Architettura a tre livelli.

File	Linguaggio	Ruolo
diagnosi_medica.pl	Prolog	Knowledge Base + regole + test
interfaccia.py	Python	GUI Tkinter + ponte verso swipl

Tabella 2 — File del progetto.

4. Knowledge Base in Prolog

La KB usa **fatti atomici** (un fatto per ogni coppia malattia–sintomo, descrizione, trattamento, categoria) più un predicato derivato *malattia/1*. È la rappresentazione più naturale per Prolog: il motore accede ai sintomi di una malattia con *findall/3* e aggiungere un sintomo significa aggiungere una sola riga.

Predicato	Descrizione	Esempio
sintomo/2	Associa una malattia a un sintomo.	sintomo(influenza, febbre_alta).
categoria/2	Categoria medica della malattia.	categoria(influenza, respiratoria).
descrizione/2	Descrizione testuale.	descrizione(influenza, '...').
trattamento/2	Trattamento raccomandato.	trattamento(influenza, '...').
malattia/1	Predicato derivato (esistenza).	malattia(X) :- categoria(X, _).

Tabella 3 — Predicati della Knowledge Base.

```
prolog

%% Sintomi: un fatto per ogni coppia (malattia, sintomo).
sintomo(influenza, febbre_alta).
sintomo(influenza, tosse_secca).
sintomo(influenza, mal_di_testa).
%% ...

%% Categorie.
categoria(influenza, respiratoria).
categoria(covid19, respiratoria).

%% Predicato derivato.
malattia(X) :- categoria(X, _).
```

Listato 1 — Estratto della Knowledge Base.

5. Regole di Inferenza

Prolog è un linguaggio di **programmazione logica** basato sulle clausole di Horn. Un programma Prolog è una *knowledge base* di fatti e regole; un'interrogazione (*query*) viene risolta dal motore inferenziale tramite due meccanismi fondamentali: l'**unificazione** (cerca una sostituzione di variabili che renda due termini sintatticamente identici) e la **risoluzione SLD** (sceglie una clausola la cui testa unifichi con il goal corrente e sostituisce il goal con il body della clausola).

Questa strategia è detta **goal-driven** o *backward chaining*: si parte dalla conclusione da dimostrare e si procede all'indietro verso i fatti. È l'opposto del *forward chaining*, che parte dai fatti e ne deriva conseguenze. Per la diagnosi medica il backward chaining è naturale: l'utente domanda «quali malattie sono compatibili con i sintomi X, Y, Z?» e il motore lavora a ritroso per verificare ogni candidato.

5.1 La regola principale: diagnosi/3

Nota sulla notazione. In Prolog ogni predicato è identificato dalla coppia *nome/arità*, dove l'*arità* è il numero di argomenti. Così *diagnosi/3* è una regola a 3 argomenti, *spiega_diagnosi/4* ne ha 4, *findall/3* è il built-in *findall* a 3 argomenti. Lo stesso nome con arità diversa indica predicati distinti.

La regola **diagnosi/3** implementa il calcolo dell'Indice di Copertura Sintomatica (ICS) per una malattia, dato l'insieme dei sintomi osservati nel paziente:

```
prolog

%% diagnosi(+SintomiPaziente, ?Malattia, -Certezza)
diagnosi(SintomiPaziente, Malattia, Certezza) :-
    malattia(Malattia),
    sort(SintomiPaziente, SintomiUnici),
    findall(S, sintomo(Malattia, S), TuttiSintomi),
    intersection(SintomiUnici, TuttiSintomi, SintomiComuni),
    length(SintomiComuni, NComuni),
    NComuni > 0,
    length(TuttiSintomi, NTotali),
    Certezza is round((NComuni / NTotali) * 100).
```

Listato 2 — La regola diagnosi/3.

La testa *diagnosi(SintomiPaziente, Malattia, Certezza)* dichiara una relazione fra tre argomenti, con modi + (input), ? (libero in input/output) e - (output). Il body è una congiunzione di sotto-goal che Prolog risolve nell'ordine scritto, sfruttando i built-in standard:

- **malattia(Malattia)** — unifica *Malattia* con un identificatore di malattia esistente nella KB. Tramite *backtracking* il motore prova tutte le malattie alternative, generando una soluzione per ognuna.
- **sort/2** — ordina la lista dei sintomi del paziente eliminando i duplicati. La regola è quindi *idempotente* rispetto alle ripetizioni dell'utente.
- **findall/3** — raccoglie in una lista *tutti* i termini *S* che rendono vero *sintomo(Malattia, S)*. È il meta-predicato canonico per materializzare una relazione come collezione.
- **intersection/3 e length/2** — calcolano l'insieme dei sintomi comuni e la sua cardinalità, secondo la definizione di ICS.
- **NComuni > 0** — vincolo che *taglia* le malattie senza alcun sintomo in comune: il motore fallisce immediatamente e fa *backtracking* sulla scelta di *Malattia*.
- **is/2** — valuta l'espressione aritmetica a destra e unifica il risultato con *Certezza*. È l'unico punto in cui Prolog si comporta in modo «procedurale».

5.2 Esempio di risoluzione passo-passo

Supponiamo che la query sia *diagnosi([febbre_alta, tosse_secca, stanchezza], influenza, C)* e che nella KB siano definiti 7 sintomi per l'influenza. Il motore esegue questi passi:

1. Unifica *Malattia* con *influenza* tramite *malattia(influenza)*, che riesce perché *categoria(influenza, respiratoria)* è un fatto della KB.
2. *sort* normalizza la lista in *[febbre_alta, stanchezza, tosse_secca]*.
3. *findall* interroga ripetutamente *sintomo(influenza, S)* e raccoglie i 7 sintomi tipici dell'influenza in *TuttiSintomi*.
4. *intersection* calcola *SintomiComuni = [febbre_alta, stanchezza, tosse_secca]* (3 elementi).
5. $NComuni = 3$, $NTotali = 7$, dunque *Certezza is round(3/7 × 100) = 43*. La query ha successo con $C = 43$.

5.3 Spiegazione della diagnosi: spiega_diagnosi/4

Un sistema esperto, secondo la definizione classica, non si limita a produrre una conclusione: deve essere capace di **spiegare** il proprio ragionamento. In Prolog questo è particolarmente naturale, perché la stessa knowledge base che ha generato il risultato può essere interrogata per scomporlo. La regola **spiega_diagnosi/4** fornisce esattamente questa explanation facility, dividendo i sintomi della malattia in *trovati* (intersezione con il quadro del paziente) e *mancanti* (differenza insiemistica):

```
prolog

%% spiega_diagnosi(+SintomiPaziente, +Malattia, -Trovati, -Mancanti)
spiega_diagnosi(SintomiPaziente, Malattia, Trovati, Mancanti) :-
    sort(SintomiPaziente, SintomiUnici),
    findall(S, sintomo(Malattia, S), Tutti),
    intersection(SintomiUnici, Tutti, Trovati),
    subtract(Tutti, Trovati, Mancanti).
```

Listato 3 — La regola spiega_diagnosi/4.

Le due liste vengono poi mostrate nella GUI per ogni diagnosi selezionata, rendendo trasparente all'utente *perché* il sistema propone quella conclusione e quali sintomi mancanti potrebbero rafforzarla o smentirla.

6. Interfaccia Grafica (Python / Tkinter)

La GUI è organizzata in tre aree affiancate: selezione sintomi (sinistra), risultati come card cliccabili ordinate per ICS (centro), dettagli della diagnosi selezionata (destra). L'utente spunta i sintomi, clicca «Analizza Sintomi» e ottiene le diagnosi candidate; cliccando su una card vede sintomi trovati e mancanti, descrizione e trattamento.

Il codice in *interfaccia.py* è organizzato in due classi: **MedExpertApp** costruisce la finestra Tkinter e gestisce eventi e stato; **PrologEngine** fa da ponte verso *swipl* via *subprocess*, invia le query e fa il parsing dell'output testuale. La diagnosi viene lanciata in un thread separato per non bloccare la UI.

7. Test e Validazione

La validazione è affidata a una suite di **7 test automatizzati** in Prolog, integrati nel file della KB. Coprono i casi rilevanti: corrispondenza esatta e parziale, diagnosi multiple, sintomi inesistenti, duplicati, spiegazione, ordinamento. Si lancia con:

```
swipl -g "test_diagnosi, halt" diagnosi_medica.pl
```

#	Nome	Scopo	Esito atteso
1	Corrispondenza esatta	Tutti e 7 i sintomi dell'influenza in input	ICS(influenza) = 100%
2	Corrispondenza parziale	3 sintomi su 7 dell'influenza	ICS = round(3/7·100) = 43%
3	Diagnosi multiple	Sintomi condivisi tra più malattie	≥ 2 diagnosi restituite
4	Nessuna corrispondenza	Sintomi non presenti nella KB	0 diagnosi
5	Gestione duplicati	Stesso sintomo ripetuto nell'input	ICS invariata
6	Spiegazione	spiega_diagnosi/4 con 2 sintomi dell'influenza	2 trovati, 5 mancanti
7	Ordinamento	diagnosi_ordinate/2 con 4 sintomi	Lista ordinata per ICS decrescente

Tabella 4 — Suite di test.

Output reale: RIEPILOGO: 7/7 superati, 0 falliti. La suite serve anche come **regression test**: ogni modifica alla KB viene verificata in automatico.

8. Conclusioni

MedExpert AI applica i temi centrali del corso — rappresentazione della conoscenza in una KB, inferenza nella logica del primo ordine con *backward chaining*, Prolog come linguaggio basato su clausole di Horn — a un dominio concreto e realistico. Combinando la KB Prolog con una GUI Python, il sistema mostra che l'AI simbolica resta uno strumento efficace quando spiegabilità, trasparenza e determinismo sono requisiti imprescindibili, come nella diagnostica medica.

Luciano Meccariello N46007465

Gaspere Tortora N46007942

Andrea Francesco Bruno N46007639